

# Adaptive Retrieval for Repository-Level Code Completion

Project status report — phases 0–5 implemented

This report documents the design and implementation status of a research project that adds a static-analysis “second-chance” gate to the CARD framework (Zhang et al. 2024) for adaptive retrieval-augmented code completion. The hypothesis: CARD’s logit-based uncertainty signal misses a class of failures — confidently-generated identifiers that don’t actually exist in the repository. Static analysis of the model’s prediction can detect these directly and trigger retrieval, reducing hallucinations without spending the full always-retrieve budget.

## What this report covers

- Research questions and the proposed cascade architecture.
- The five implementation phases completed so far (setup, static analysis, generator/retriever/baselines/metrics, CARD reimplementation, cascade integration, evaluation infrastructure).
- Key design decisions made during implementation, including deviations from the original guide and the reasoning behind them.
- Current test and validation status (151 passing tests, 5/5 validation gates passed locally).
- What remains: the actual experiment runs (Phase 6), analysis (Phase 7), and paper writing (Phase 8).

# 1. Project goals and research questions

The project studies whether **adaptive retrieval** — selectively triggering RAG only on instances where it would help — outperforms always-retrieve and never-retrieve baselines for repository-level code completion. The novel contribution is adding a static-analysis cascade stage to CARD's decision flow.

## Hypothesis

CARD's uncertainty signal — aggregated per-token probability and entropy from the generator's logits — misses a specific failure mode: confidently-generated identifiers that don't actually exist anywhere in the repository. Static analysis of the prediction's abstract syntax tree can detect these by checking whether each used identifier resolves in the file's scope, in the repository's symbol table, or as a Python builtin. When an identifier resolves nowhere, the prediction is almost certainly a hallucination, and retrieval should fire even if CARD's probability signal looked confident.

## Research questions

| RQ  | Question                                                                                                                                                                                      | Hypothesis                                                                                                                          |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| RQ1 | How does the cascade compare to always-retrieve and never-retrieve baselines on CrossCodeEval-Python and RepoEval, by accuracy (EM, ES, Identifier-F1) and efficiency (% retrieval, latency)? | H1: cascade matches or beats always-retrieve on accuracy while performing fewer retrievals.                                         |
| RQ2 | Does the static-analysis stage reduce the rate of identifier hallucinations versus vanilla CARD?                                                                                              | H2: cascade cuts per-instance hallucination rate by at least 20% relative to CARD, significant under McNemar's test ( $p < 0.05$ ). |
| RQ3 | How does vanilla CARD compare against always- and never-retrieve on the same datasets?                                                                                                        | H3: CARD matches always-retrieve in accuracy (within $\pm 1$ ES) while saving 20–46% of retrievals, replicating the CARD paper.     |

RQ3 exists because the CARD paper compares CARD against iterative-RAG systems (RepoCoder), not directly against the always/never baselines the project description requires.

## 2. System architecture

### The cascade

The pipeline is three asymmetric stages: zero-shot generation, CARD's uncertainty gate, and then a static-analysis gate that runs *only when CARD says skip*. The asymmetry is deliberate: static analysis can only *add* retrievals to CARD's decisions, never remove them. This bounds the worst-case retrieval count to “always-retrieve” and frames the central question cleanly as “does the extra retrieval budget reduce hallucinations?” rather than a confounded accuracy-vs-cost tradeoff.

```
Stage 1. y0 <- Generator(x_left, x_right)           # zero-shot
Stage 2. if CARD.is_retrieve(y0, ...) :             # CARD's signal
        y <- Generator(x_left, x_right, BM25-chunks)
        return y, reason='card'
Stage 3. sa <- StaticAnalyzer(y0, x_left, x_right) # the new gate
        if sa.fires:
            y <- Generator(x_left, x_right, BM25-chunks)
            return y, reason='static_unresolved' or 'static_crossfile'
        return y0, reason='none'
```

### Six experimental configurations

| ID | Name                 | Behaviour                                                                                                                  |
|----|----------------------|----------------------------------------------------------------------------------------------------------------------------|
| C1 | no-retrieve          | Zero-shot only.                                                                                                            |
| C2 | always-retrieve      | BM25 top-10, then generate with the chunks prepended.                                                                      |
| C3 | CARD (single-RAG)    | Zero-shot, then RAG iff CARD's gate fires.                                                                                 |
| C4 | cascade (ours)       | Zero-shot, then RAG iff CARD's gate <i>or</i> the static-analysis gate fires.                                              |
| C5 | static-only ablation | Zero-shot, then RAG iff static analysis alone fires.                                                                       |
| C6 | oracle               | Generate both no-retrieve and always-retrieve; keep the one with the higher ES against the ground truth. Upper bound only. |

### The novel static-analysis signal

For each prediction, the analyzer extracts every used identifier from its AST and classifies it as one of:

- **builtin** — Python built-in or visible at the hole. Not interesting.
- **in-file** — in scope at the hole (imports, parameters, module-level defs, prior bindings). Not interesting.
- **cross-file** — in the repository's symbol table but not visible in-file. Fires retrieval — bringing this file into context would help.
- **unresolved** — nowhere. Fires retrieval — this is almost certainly a hallucination.

Both fire-on-crossfile and fire-on-unresolved are configurable flags for the ablation matrix. Within stage 3, 'unresolved' wins as the trigger reason when both fire on the same prediction — it's the stronger hallucination signal.

### 3. Datasets

| Dataset                                                         | Role                                                                             | Size                | Status                                                    |
|-----------------------------------------------------------------|----------------------------------------------------------------------------------|---------------------|-----------------------------------------------------------|
| CrossCodeEval-Python (Ding et al. 2023)                         | Primary evaluation. Has Identifier-F1 metric and explicit cross-file annotation. | 2,665 instances     | Downloaded, loader verified                               |
| RepoEval-line / -api / -function (Zhang et al. 2023, RepoCoder) | Secondary evaluation. Used for CARD reproduction gate.                           | 1,600 / 1,600 / 373 | Loader written; data not yet downloaded                   |
| The Stack (subset) (bigcode/the-stack-smol)                     | Source of (X, y) pairs for training the CARD Estimator.                          | ~10k Python files   | Loader integrated in train_data.py; full run is GPU-bound |

#### Schema deviations from the implementation guide

The original guide documented CrossCodeEval's record shape from the paper, but the shipped data's schema differs in three places, all documented inline in the loader:

- The raw line\_completion.jsonl has crossfile\_context: None. The chunks live only in the rg1\_\* / oracle\_\* variants. Since prompt / groundtruth / right\_context are byte-identical across all variants, we default to line\_completion\_rg1\_bm25.jsonl to borrow the chunk list — this does *not* inject retrieval into the prompt.
- crossfile\_context is a dict with a list key wrapping the chunk array, not a bare list as the guide's example showed.
- repository and task\_id are nested under metadata, not top-level.

## 4. Implementation phases

The build order is risk-front-loaded: anything that could blow up the schedule (data acquisition, CARD reproducibility, GPU budget) gets touched early or has a validation gate that catches issues before they compound.

### Phase 0 — setup

Conda environment with Python 3.11.15, directory scaffolding, requirements-dev.txt with the macOS-installable subset of dependencies (excludes vLLM which is Linux/CUDA-only). Tree-sitter pins bumped from the guide's 0.21.0 to 0.23.x because the older version has no Python 3.11 wheels on PyPI — the 0.23.x API matches the code in the guide.

### Phase 1 — static analysis

The novel contribution. Four modules built in order:

- `parser.py` — tree-sitter setup.
- `symbol_table.py` — collects function/class/assignment names across all .py files in a repository, with a common-name filter (excludes single-letter names and ~30 short stopwords like result, tmp) to avoid spurious cross-file matches. Constructible from a filesystem path or an in-memory {filename: content} dict (the latter is what CrossCodeEval needs).
- `scope.py` — given a Python source string and a byte position, returns the set of names visible at that point: imports (including aliased and from X import Y forms), module-level defs, function parameters, for-loop and comprehension variables, with-aliases, walrus targets, lambda parameters, splat parameters.
- `analyzer.py` — the prediction-side classifier. Extracts every used identifier from the prediction's AST, subtracts identifiers bound within the prediction itself (lambda params, comprehension vars), and classifies the remainder.

**Validation:** 22 edge-case tests from Appendix E of the guide, all passing including the soft-xfail cases for walrus and type hints (which work cleanly thanks to named\_expression handling).

**Notable bug found and fixed:** tree-sitter 0.23.x's `child_by_field_name` returns a fresh Python wrapper on each call, so the is identity comparison fails (returns False for the same logical node). Switched to `.id ==` comparison. Without this, every attribute name (`np.array`'s `array`, `self.x`'s `x`, `f.read()`'s `read`) was being falsely flagged as unresolved.

### Phase 2 — generator, retriever, prompts, baselines, metrics

- **Generator:** HFGenerator (HuggingFace transformers, works on Mac MPS / CPU / Linux CUDA), VLLMGenerator (Linux+CUDA only), MockGenerator for unit tests, `make_generator()` factory with auto-detect.
- **Retriever:** BM25 with 20-line / stride-10 chunking matching the RepoCoder convention. `make_query()` helper shared across baselines / CARD / cascade so every retrieval call uses the same query.
- **Prompt assembly:** FIM templates for Qwen / CodeLlama / StarCoder token families, with retrieved chunks prepended as commented context.
- **Baselines:** `no_retrieve_baseline` (C1), `always_retrieve_baseline` (C2).

- **Metrics:** EM, ES, Identifier-F1, repository\_symbol\_precision, hallucination\_flag, % retrieval, mean latency, paired McNemar exact-binomial test, paired bootstrap.
- **Dataset loader:** load\_crosscodeeval\_python(), verified on the real shipped JSONL.

**Validation gate:** 50 CCE-Python instances run through both baselines end-to-end; all metrics compute without errors.

### Phase 3 — CARD reimplementation

- features.py — CARD's 13-D feature vector (Table 1 of Zhang et al. 2024). Six statistics each over per-token probabilities and entropies (max, min, avg, std, prod, geomavg), plus generation length. Product and geometric average computed in log-space to avoid float underflow on long sequences; output clipped to float32 range before downcast (the entropy product can overflow float32 for moderate-length high-entropy sequences).
- estimator.py — LightGBM regressor wrapper, training with early stopping. Plus MockEstimator returning scripted s-hat values for unit tests.
- pipeline.py — CARD's Algorithm 1, single-RAG variant. is\_retrieve gate, select step that compares zero-shot vs RAG output ratios.
- train\_data.py — the (X, y) sampling, K-means deduplication, and generator orchestration for building the Estimator's training set. Helpers are pure-Python and unit-tested; the full ~250k-pair generation step is the only GPU-bound piece.

**Validation gates:** feature-extraction shape and finite-output checks across random inputs; Estimator MSE on synthetic data with known signal hit 0.0014, well under the paper-target 0.10 (paper reports ~0.07 with CodeLlama-7B). The third gate — reproducing the paper's Table 3 numbers within  $\pm 1\%$  ES on RepoEval-line — requires GPU and the original CodeLlama-7B and is the only Phase 3 gate not yet met.

### Phase 4 — cascade integration

Pure wiring of pieces from phases 1–3. The cascade\_pipeline function and CascadeOutput dataclass implement the three-stage flow described in Section 2. An 'exploding-analyzer' test verifies that the static-analysis stage is *not* consulted when CARD's gate has already fired, preserving the asymmetric cascade invariant.

**Validation gate:** cascade run on 20 real CrossCodeEval-Python instances exercises 3 of 4 trigger reasons (none, card, static\_unresolved) with a mock generator. The fourth (static\_crossfile) is covered deterministically by a unit test — it requires the prediction to use a name that happens to exist in repo\_files but not in-file, which is rare with uniform mock predictions but normal with a real model. The asymmetric-cascade invariant cascade\_retrieved  $\geq$  card\_alone\_retrieved holds on every instance.

### Phase 5 — evaluation infrastructure

- CachedGenerator — disk-backed cache around any Generator, keyed on sha256(model :: prompt :: max\_tokens). generate\_batch smart-routes: cache hits skip the inner generator entirely, only misses get batched.
- eval/runner.py — run\_experiment(config, dataset, ...) dispatches all six configs and writes the \$15.1 per-instance JSONL; aggregate\_from\_jsonl() recomputes the \$15.2 summary without re-running anything.

- `load_repoeval()` — RepoEval loader for line / api / function tasks. Raises a clear error with download instructions until the user grabs `datasets.zip` and `repositories/` from the RepoCoder GitHub.
- `scripts/04_run_experiment.py` — the CLI: `--config × --dataset × --backend {mock,hf,vllm} × --estimator-path × --cache-dir × --limit`.

**Validation gate:** 50 CrossCodeEval-Python instances × all 6 configs ran end-to-end via the CLI. After C1+C2 warmed the cache (100 generations), C3/C4/C5/C6 each hit **100% cache hit rate** — 100 hits per config, 0 misses. With a real model this is the difference between hours and minutes for the full matrix.

## 5. Key design discussions

### 5.1 Why vLLM specifically?

The original guide pins vLLM and treats it as required. The reality is that vLLM is a *performance* choice, not architectural — the Generator interface returns the same dataclass regardless of backend. vLLM brings three things:

- Batched throughput: PagedAttention + continuous batching are 10–20× faster than naive HF transformers on the same GPU. The 90k-generation main matrix and the 250k-pair Estimator training set basically require this.
- Clean per-token top-K logprob extraction via SamplingParams(logprobs=K) — needed for CARD’s 13-D feature vector.
- Deterministic greedy decoding consistent across batch sizes, which matters for the generation cache.

We added an HFGenerator backend that works on Mac (MPS/CPU) and CUDA. It produces the same Generation shape, just slower. This is enough to unblock development locally; the GPU node still wants vLLM for the production runs.

### 5.2 macOS development deviations

Two adjustments were needed to make the local environment work:

- **vLLM split**: requirements-dev.txt excludes vLLM (Linux/CUDA only). requirements.txt retains it for the GPU node. Both share a tree-sitter bump from the guide’s 0.22.3/0.21.0 to 0.23.2/0.23.6 because the older tree-sitter-python has no Python 3.11 wheels on PyPI.
- **The relaxed Phase 0 gate**: the guide’s strict check is `import vllm; import lightgbm; import tree_sitter_python`. Locally we run a relaxed variant that skips vLLM and verifies the 0.23.x tree-sitter API works exactly as the guide’s code expects.

### 5.3 Model-size risks

We discussed running locally with a smaller model (Qwen2.5-Coder-0.5B or 1.5B) instead of the paper’s 7B. Real risks, by category:

#### ***Internal validity of CARD***

Smaller models are notoriously miscalibrated — high token-probability doesn’t reliably correlate with correctness. CARD’s Estimator trains on (features, ES) pairs, so miscalibration directly shows up as higher MSE: the paper reports ~0.07 with CodeLlama-7B; 0.5B is likely to land at 0.10–0.18. As MSE rises, `is_retrieve` and select decisions get noisier and CARD approaches “retrieve at random”, at which point it may not beat always-retrieve.

#### ***Baseline landscape***

Smaller models benefit more from retrieval, so always-retrieve is a stronger baseline. The opportunity for adaptive savings shrinks. The *direction* of the cascade-vs-baselines comparison should hold; the *magnitudes* will not transfer to 7B.

#### ***Hallucination signal***

0.5B hallucinates more often, but its hallucinations are often obviously wrong (made-up tokens). 7B’s hallucinations are subtler (plausible-looking names that don’t exist). Net effect on RQ2: unknown direction, but the result wouldn’t auto-generalize.

**Reviewer-facing risks**

We lose the CARD-paper reproduction gate (it requires CodeLlama-7B specifically). A 0.5B-only paper invites a “does this generalize?” pushback. The mitigation is reserving a small (~200-instance) GPU-node validation slice with 7B that confirms trends from the local 0.5B/1.5B run.

**5.4 Local vs GPU-node work split**

Most of the codebase doesn’t need a model and can be built locally:

| Local-buildable                                                                                                                                                                                                                                           | GPU-node-only                                                                                                                                                                     |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| All of static analysis<br>BM25 retriever, prompt assembly<br>CARD features, Estimator (trains on cached arrays)<br>CARD pipeline, cascade pipeline<br>Metrics, statistical tests<br>Dataset loaders, runner, generation cache<br>Phase 7 analysis scripts | Generator <i>execution</i> at scale<br>CARD Estimator training data (250k generations)<br>CARD reproduction gate vs paper Table 3<br>Phase 6 experiment matrix (~90k generations) |

The mlx-lm option (Apple’s native ML framework) gives 2–3× faster generation than HF transformers on Apple Silicon and would make the full pipeline locally runnable end-to-end at small scale. Adding it is queued and was discussed but not yet built.

## 6. Current status

### 6.1 Test coverage

| Test file                        | Count      | Focus                                       |
|----------------------------------|------------|---------------------------------------------|
| test_static_analysis.py          | 3          | Basic static-analysis tests (§10.5)         |
| test_static_analysis_extended.py | 22         | Edge cases from Appendix E                  |
| test_retriever.py                | 11         | BM25 chunking, ranking, edge cases          |
| test_prompt.py                   | 8          | FIM template assembly per family            |
| test_baselines.py                | 4          | C1/C2 with MockGenerator                    |
| test_metrics.py                  | 18         | Accuracy, hallucination, McNemar, bootstrap |
| test_datasets.py                 | 7          | CrossCodeEval real-JSONL loader             |
| test_features.py                 | 12         | CARD 13-D vector, log-space stability       |
| test_estimator.py                | 11         | Train/predict/save/load + MockEstimator     |
| test_card_pipeline.py            | 11         | Both CARD branches, latency, defaults       |
| test_train_data.py               | 13         | Filtering, sampling, K-means dedup          |
| test_cascade.py                  | 10         | All 4 trigger reasons + precedence          |
| test_cache.py                    | 8          | Hit/miss, persistence, batch routing        |
| test_runner.py                   | 13         | One per config + schema + aggregates        |
| <b>Total</b>                     | <b>151</b> | <b>All passing, lint clean</b>              |

### 6.2 Validation gates

| Gate                                                       | Status                              |
|------------------------------------------------------------|-------------------------------------|
| Phase 0: deps importable (relaxed, vLLM deferred)          | PASS                                |
| Phase 1: all 22 Appendix-E static-analysis tests pass      | PASS                                |
| Phase 2: 10 CCE instances × baselines, metrics no errors   | PASS                                |
| Phase 3a: feature-vector shape and stability               | PASS                                |
| Phase 3b: Estimator MSE on synthetic data < 0.10           | PASS (0.0014)                       |
| Phase 3c: CARD vs paper Table 3 within ±1% ES              | deferred (needs GPU + CodeLlama-7B) |
| Phase 4: cascade exercises 2+ trigger reasons on real data | PASS (3/4 with mock)                |
| Phase 5: 50 instances × 6 configs end-to-end               | PASS                                |

### 6.3 Code organisation

```
src/adaptive_retrieval/  
  generator.py      # HF / vLLM / Mock / Cached backends  
  retriever.py      # BM25 + make_query helper  
  prompt.py         # FIM templates for qwen/codellama/starcoder  
  baselines.py      # C1, C2
```

```

cascade.py          # C4: CARD + static-analysis cascade
metrics.py          # EM, ES, IdF1, hallucination, McNemar, bootstrap
card/
  features.py        # 13-D Table 1 vector
  estimator.py        # LightGBM wrapper + MockEstimator
  pipeline.py         # Algorithm 1, single-RAG
  train_data.py       # Stack sampling + K-means dedup
static_analysis/
  parser.py           # tree-sitter setup
  symbol_table.py     # repo-wide name table, filesystem or in-memory
  scope.py            # InFileScopeAnalyzer.visible_at(source, hole_byte)
  analyzer.py         # PredictionAnalyzer (the novel signal)
eval/
  datasets.py         # Instance, load_crosscodeeval_python, load_repoeval
  runner.py           # run_experiment, aggregate_from_jsonl
scripts/
  02_smoke_generator.py # HF backend smoke (downloads a small model)
  03_smoke_pipeline.py  # 10 CCE instances via both baselines
  04_run_experiment.py  # CLI for any config x dataset combo
  04_smoke_card.py      # CARD smoke with synthetic Estimator
  05_smoke_cascade.py   # cascade smoke with trigger-distribution report
tests/                 # 151 tests, all passing

```

## 7. What remains

### Phase 6 — main experiments

Run the matrix: 6 configs × CrossCodeEval-Python (2,665 instances) × 1 model, plus the same configs on RepoEval-line and RepoEval-API (1,600 each), plus the ablations on CrossCodeEval-Python. Roughly 90k generations end-to-end with the generation cache active.

- Locally feasible with Qwen2.5-Coder-0.5B or 1.5B via mlx-lm (~8–20 hours).
- GPU-feasible with Qwen2.5-Coder-7B or CodeLlama-7B on a single A100 (~12–48 hours).
- Cache writes are immutable across configs, so threshold sweeps and ablations re-run for the cost of metrics, not generations.

### Phase 7 — analysis

- Per-trigger-reason breakdown of the cascade.
- Disagreement analysis: instances where CARD says no but static fires.
- McNemar test on hallucination rate (CARD vs cascade).
- Paired bootstrap CI on ES differences.
- Threshold sweep plots for T\_RAG.

All operations on the JSONL results files — no GPU needed. The statistical tests are already implemented and unit-tested in metrics.py.

### Phase 8 — paper writing

8–10 page two-column paper. Outline: introduction (1 pg), background and related work (1 pg), method — CARD recap + the cascade and static-analysis signal (2 pg), experiments — config table, headline results, ablations (3 pg), analysis — disagreement analysis, qualitative examples (2 pg), discussion and limitations (0.5 pg), conclusion (0.5 pg).

### Immediate next decisions

- **mlx-lm backend**: ~150 lines to add, would unblock Phase 6 locally. Discussed in detail, ready to implement.
- **RepoEval download**: needed for the CARD reproduction gate and the secondary evaluation tables. The loader is ready.
- **Estimator training data**: the 24-hour bottleneck. With mlx-lm and Qwen2.5-Coder-0.5B, ~5–10 hours for 50k pairs locally.

End of report.